Document 2: Benchmarks

Kit: qae_european 1.0.0 · **Scope:** the classical methods amplitude estimation is compared with: which ones, why, how they are implemented, with which parameters, and which alternatives were considered and not used.

Companion documents: Document 1: Circuit · Document 3: Tests · User guide.

This file is generated by `docs/build_benchmarks_doc.py` from `docs/benchmarks_head.md` and the source files, so the code in section 6 is always the code that ran. Edit the head file or the code, not this file.

---

1. What a benchmark must answer

The method under test reads a price from an amplitude that it estimates with a number of **calls** to the circuit A. A benchmark is useful if it answers one of three questions:

1. **What is the right answer?** A reference price with no sampling error.
2. **Is the quantum estimator better than the classical sampler it is meant to replace,** measured on the same target and with the same number of calls?
3. **How does it compare with the best classical sampling method** for this problem, not just the plain one?

2. How performance is measured

The full definition is in Document 3, section 1.1. In short:

- **Performance is measured in calls**, not in time:
  - for amplitude estimation, one application of A;
  - for Monte Carlo, one simulated path;
  - for quasi-Monte Carlo, one point of the sequence.
- **Error is the root-mean-square error (RMSE)** over independent repetitions, against a stated target. The median absolute error is also reported.
- **Two methods are compared by their relative efficiency** E(ε) = N_comparator(ε) / N_QAE(ε), the ratio of calls each needs to reach the same RMSE ε.

Wall-clock time is deliberately out of scope. A classical simulation of a quantum circuit is far slower than Monte Carlo on the same machine, and a timing of the simulator says nothing about hardware.

3. The benchmarks chosen

Benchmark	Target	Question it answers	Class
Black-Scholes closed form	Continuous price	1. The reference	`BlackScholes`
Plain Monte Carlo on the grid	Grid price	2. Like-for-like comparator	`MonteCarlo("grid", n)`
Plain Monte Carlo, continuous	Continuous price	2. What a desk would get by plain sampling, including all errors	`MonteCarlo("continuous")`
Quasi-Monte Carlo, scrambled Sobol	Continuous price	3. Best classical sampling method	`QuasiMonteCarlo`

All of them share one interface, `price(option, budget, rng)`, which returns an `Estimate(value, calls, stderr)`.

3.1 Black-Scholes closed form: the reference

$C = S_0\,N(d_1) - K\,e^{-rT}\,N(d_2)$, with $d_1 = [\ln(S_0/K) + (r + \sigma^2/2)\,T] / (\sigma\sqrt{T})$ and $d_2 = d_1 - \sigma\sqrt{T}$.

- **Why.** The model is Black-Scholes, so the closed form is exact: every error against it is a true error of the method. It uses no samples (calls = 0).
- **Accuracy.** `scipy.stats.norm.cdf` is accurate to machine precision over the campaign's ranges. The smallest in-scope price is 0.01, well above any rounding issue.
- **Limit.** It prices the continuous distribution, not the grid the circuit sees. Errors against it mix the discretisation bias and the statistical error. That is why the like-for-like comparator below targets the grid.

3.2 Plain Monte Carlo on the grid: the like-for-like comparator

- **How.** It draws grid points i with the probabilities p_i of the circuit's `LognormalGrid`, and averages the discounted payoffs $e^{-rT}\max(x_i - K, 0)$. Drawing N points is implemented as one multinomial draw of the counts, which is equivalent and faster. The standard error is the sample standard deviation divided by $\sqrt{N}$.
- **Why.** It estimates exactly the quantity the circuit encodes, the grid price, with the classical procedure the quantum algorithm replaces: sampling the distribution and averaging the payoff. A quantum computer measuring A directly, without Grover steps, would behave the same way: Bernoulli draws with probability a. Amplitude estimation is supposed to beat that, and this comparator isolates the claim. There is no discretisation bias and no difference in target, only statistical error against statistical error, at the same number of calls.
- **Used for:** H1 and H2 (Document 3).

3.3 Plain Monte Carlo, continuous

- **How.** $S_T = S_0 \exp((r - \sigma^2/2)\,T + \sigma\sqrt{T}\,Z)$, with Z standard normal, then the discounted average payoff. Unbiased for the Black-Scholes price.
- **Why.** It is what a desk would actually run without a quantum computer. It shows the total error a user would see, against the continuous price, with no discretisation.

3.4 Quasi-Monte Carlo, scrambled Sobol: the best classical sampler

- **How.**
  - Take a one-dimensional Sobol sequence with Owen scrambling (`scipy.stats.qmc.Sobol`, `scramble=True`).
  - Map each point u_j to $Z_j = \Phi^{-1}(u_j)$, then to S_T as in plain Monte Carlo, and average the discounted payoffs.
  - The number of points is the largest power of two not above the budget, because Sobol's balance properties hold for powers of two.
  - Each repetition uses a fresh scramble, so repetitions are independent randomisations and their spread measures the error. The method reports no standard error of its own.
- **Why.** The payoff is a smooth function of one variable, apart from the kink at the strike. In one dimension, scrambled Sobol converges at close to 1/N, against $1/\sqrt{N}$ for plain Monte Carlo. That is the same rate amplitude estimation promises. Leaving it out would compare the quantum method only with the weakest classical sampler. Document 3 states in advance (H4) that amplitude estimation is **not** expected to beat it.
- **Cost of the power-of-two rule.** At each budget, quasi-Monte Carlo uses fewer points than the others:

Budget (calls)	1,800	3,500	6,800	13,300	26,200
Sobol points	1,024	2,048	4,096	8,192	16,384

That is between 57% and 63% of the budget. This handicaps quasi-Monte Carlo, never amplitude estimation. The number of points actually used is recorded (`calls_qmc`), and relative efficiencies are computed with it.

4. Variance reduction: none in the comparators

Plain Monte Carlo is run **without** variance reduction (no antithetic variates, control variates, importance sampling or stratification). Three reasons:

1. **It is what amplitude estimation replaces.** The quadratic speed-up is stated against plain sampling. Variance reduction lowers the constant in front of $1/\sqrt{N}$ but not the rate.
2. **It applies to both sides.** The same ideas exist for amplitude estimation (for example, encoding a control variate in the payoff, or a better-adapted grid). Adding them to one side only would mix two effects.
3. **The best classical method is covered separately.** Scrambled Sobol (section 3.4) is itself a variance-reduction method, and the strongest one for this one-dimensional problem. It answers "what if the classical side tries hard".

A reader who wants Monte Carlo with variance reduction can subclass `Benchmark` (section 6) and run it through the same campaign code.

5. Benchmarks considered and not used

Candidate	Why not used
Deterministic quadrature (e.g. Gauss-Hermite, or the exact sum over the grid)	For a one-dimensional integral, quadrature beats every sampling method: it is exact to machine precision in a few dozen points. It is used, implicitly, as the reference: the grid price is that exact sum, and Black-Scholes is the exact integral. As a *comparator* it would only state that sampling is the wrong tool in one dimension. That is true and is said here, but amplitude estimation is meant for problems where quadrature fails (many dimensions, path dependence), and this kit only tests the method on the simplest case.
Monte Carlo with variance reduction	See section 4: it changes the constant, not the rate, and would have to be matched on the quantum side. Left to users through the `Benchmark` interface.
Multilevel Monte Carlo	It reduces the cost of time discretisation in path simulation. Here S_T is sampled exactly in one step, so there is nothing for it to reduce.
Other amplitude-estimation variants (canonical QAE with phase estimation, iterative QAE, power-law MLAE)	These are alternatives to the method under test, not classical benchmarks. Canonical QAE needs controlled Grover operators and deeper circuits. Iterative QAE is a natural next comparison, out of scope for the simplest kit.
An external quantum implementation (e.g. Qiskit's finance application module)	It would cross-check the circuit against an independent implementation of the same construction. Not used, to keep the dependencies to Qiskit, NumPy and SciPy. The circuit is instead checked against the exact grid price (H0-a) and by full statevector simulation (H0-b).
Wall-clock timing	Out of scope (section 2).

6. The code

The benchmarks live in `src/qae_european/benchmarks/`. Below are the full source files, comments included, exactly as run.

6.1 `base.py`: Shared interface and result type

```python
"""Shared engine of the benchmarks: the result type and the common interface."""
from dataclasses import dataclass
import numpy as np


@dataclass(frozen=True)
class Estimate:
    """Result of one pricing run.
```

```
    value : float         price estimate
    calls : int           number of payoff evaluations (paths or points) actually used; 0 for a closed
 ↪  form
    stderr : float        standard error reported by the method itself, if it has one (NaN otherwise)
    """
    value: float
    calls: int
    stderr: float = float("nan")


class Benchmark:
    """Interface: price(option, budget, rng) returns an Estimate using at most `budget` payoff
     ↪  evaluations."""
    name = "benchmark"

    def price(self, option, budget: int, rng: np.random.Generator) -> Estimate:  # pragma: no cover
        raise NotImplementedError

    @staticmethod
    def _discounted_payoffs(option, terminal_prices: np.ndarray) -> np.ndarray:
        """Shared step: discounted call payoffs for an array of terminal prices."""
        return option.discount * option.payoff(terminal_prices)
```

6.2 `black_scholes.py`: Black-Scholes closed form

```
"""Black-Scholes closed form for a European call: the reference price."""
import math
from scipy.stats import norm

from .base import Benchmark, Estimate


class BlackScholes(Benchmark):
    """C = S0 N(d1) - K exp(-rT) N(d2),
    d1 = [ln(S0/K) + (r + sigma^2/2) T] / (sigma sqrt T),  d2 = d1 - sigma sqrt T.

    Exact under the model; uses no samples (calls = 0). Implemented with scipy.stats.norm.cdf,
    which is accurate to machine precision across the parameter ranges of the campaign.
    """
    name = "black_scholes"

    @staticmethod
    def call(option) -> float:
        sq = option.log_std
        d1 = (math.log(option.spot / option.strike) + (option.rate + 0.5 * option.vol ** 2) *
 ↪  option.maturity) / sq
        d2 = d1 - sq
        return float(option.spot * norm.cdf(d1) - option.strike * option.discount * norm.cdf(d2))

    def price(self, option, budget: int = 0, rng=None) -> Estimate:
        return Estimate(self.call(option), 0, 0.0)
```

6.3 `monte_carlo.py`: Plain Monte Carlo, continuous and on the grid

```
"""Plain Monte Carlo, with two sampling settings.

sampling="continuous": draws S_T exactly from the lognormal, S_T = S0 exp((r - sigma^2/2) T + sigma
 ↪  sqrt(T) Z),
                       Z standard normal. Unbiased for the Black-Scholes price.
sampling="grid":       draws grid points with the probabilities of the circuit's LognormalGrid
                       (n price qubits). Unbiased for the grid price, i.e. the same target as the
 ↪  circuit:
                       it isolates statistical error from discretisation bias.

No variance reduction: plain sampling is exactly what amplitude estimation replaces.
Standard error = sample standard deviation / sqrt(budget).
"""
```

```python
import numpy as np

from .base import Benchmark, Estimate
from ..grid import LognormalGrid


class MonteCarlo(Benchmark):
    def __init__(self, sampling: str = "continuous", n_qubits: int | None = None):
        if sampling not in ("continuous", "grid"):
            raise ValueError("sampling must be 'continuous' or 'grid'")
        if sampling == "grid" and n_qubits is None:
            raise ValueError("grid sampling needs n_qubits")
        self.sampling, self.n_qubits = sampling, n_qubits
        self.name = "monte_carlo" if sampling == "continuous" else f"monte_carlo_grid_{n_qubits}"

    def price(self, option, budget: int, rng: np.random.Generator) -> Estimate:
        if budget < 2:
            raise ValueError("budget must be at least 2")
        if self.sampling == "continuous":
            z = rng.standard_normal(budget)
            st = np.exp(option.log_mean + option.log_std * z)
            pay = self._discounted_payoffs(option, st)
            return Estimate(float(pay.mean()), budget, float(pay.std(ddof=1) / np.sqrt(budget)))
        g = LognormalGrid(option, self.n_qubits)
        counts = rng.multinomial(budget, g.probs)             # equivalent to drawing `budget` grid
 ↪  points
        pay = option.discount * g.payoffs()
        mean = float(np.dot(counts, pay) / budget)
        var = float(np.dot(counts, (pay - mean) ** 2) / (budget - 1))
        return Estimate(mean, budget, float(np.sqrt(var / budget)))
```

6.4 `quasi_monte_carlo.py`: Quasi-Monte Carlo, scrambled Sobol

```python
"""Quasi-Monte Carlo with a scrambled Sobol sequence (Owen scrambling, scipy.stats.qmc.Sobol).

One-dimensional problem: u_j in (0,1) from the Sobol sequence, Z_j = Phi^{-1}(u_j), then S_T as in
 ↪  plain
Monte Carlo. The number of points is the largest power of two not above the budget (Sobol's balance
properties hold for powers of two), so `calls` can be below the budget; the campaign records it.

Each call uses a fresh scramble seeded from `rng`, so repeated runs are independent randomisations:
the campaign measures the error across repetitions. The method itself reports no standard error.
"""
import numpy as np
from scipy.stats import norm, qmc

from .base import Benchmark, Estimate


class QuasiMonteCarlo(Benchmark):
    name = "quasi_monte_carlo"

    def price(self, option, budget: int, rng: np.random.Generator) -> Estimate:
        if budget < 2:
            raise ValueError("budget must be at least 2")
        m = int(np.floor(np.log2(budget)))
        sobol = qmc.Sobol(d=1, scramble=True, seed=rng)
        u = sobol.random_base2(m).ravel()
        st = np.exp(option.log_mean + option.log_std * norm.ppf(u))
        pay = self._discounted_payoffs(option, st)
        return Estimate(float(pay.mean()), 2 ** m)
```

7. Parameters used in the campaign

Parameter	Value	Where
Budgets (calls)	1,800 · 3,500 · 6,800 · 13,300 · 26,200 (equal to the amplitude-estimation ladders of depth 4 to 8)	`campaign.py`
Sobol points	largest power of two not above the budget	`quasi_monte_carlo.py`
Repetitions per configuration	50, independent	`campaign.DRAWS`
Seeds	master seed 20261103; one stream per (scenario, qubits, depth, method) through `numpy.random.SeedSequence`	`campaign.rng_for`
Variance reduction	none in Monte Carlo; Owen scrambling in quasi-Monte Carlo	section 4
Grid for Monte Carlo on the grid	the circuit's own `LognormalGrid(option, n)`, width 4 standard deviations	`monte_carlo.py`, `grid.py`

Continuous Monte Carlo and quasi-Monte Carlo do not depend on the number of qubits. They are computed once per scenario and budget, and repeated on each qubit row of the results.

8. Unit tests of the benchmarks

`tests/test_package.py` checks:

- the Black-Scholes price against the textbook value ($S_0 = K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$: 10.4506) to 10^{-9}, and the no-arbitrage bounds on a second case;
- that both Monte Carlo settings are unbiased: with 200,000 paths, within 4 standard errors of their own target (Black-Scholes, grid price);
- that quasi-Monte Carlo uses 4,096 points for a budget of 5,000 and lands within 0.02 of Black-Scholes;
- that a campaign row is identical when recomputed from its seed.

Run them with `pytest tests`.